



**KARNATAK UNIVERSITY'S
KARNATAK ARTS/SCIENCE COLLEGE, DHARWAD**

Department of Mathematics
In Association with
Karnatak University's Mathematics Teachers Association (KUMTA)

Organizes
One-Day Workshop on GNU Octave Programming

**LABORATORY MANUAL
FOR MATHEMATICS
PRACTICALS**

Skill Enhancement Course (SEC)

For B.Sc. IV/V/VI Semester
(Revised NEP)
(As per Karnatak University Syllabus)

Prepared by

Dr. Prabha S. Mandalageri

Head, Dept of Mathematics

&

Mr. Akash T. Naik

KLEs S.K. Arts College and
H.S.K. Science Institute, Hubballi

Effective from
the academic year 2025-26

CONTENTS

LAB	List of Experiments	Page No.
*	Introduction to GNU Octave	1
1	Program to compute addition and subtraction of matrices	4
2	Program to compute rank of matrix and row reduced echelon form.	5
3	Program to find the angle between the radius vector and tangent	6
4	Program to find the nth derivative of e^{ax} and trigonometric functions	7
5	Program to construct Cayley table and test Commutative for given finite set	8
6	Programs to verify Lagrange's theorem (groups) with suitable example	10
7	Program to evaluate the double and triple integrals with constant limits	11
8	Program to verify the exactness of a differential equation	12
9	Program to find the Complementary Function and Particular Integral of linear differential equations with constant coefficients.	13
10	Program to verify D'Alembert's and Raabe's tests with suitable examples	14
11	Program to find the Laplace transforms of some standard functions.	15
	Program to find the Laplace transforms of periodic functions.	16
12	Program to find sine and cosine Fourier transforms	17
**	Summary of commands	18

Introduction to GNU Octave

GNU Octave is a high-level, open-source numerical computing software primarily designed for mathematical computations, data analysis, and scientific simulations. It is widely used in the fields of engineering, mathematics, science, and education. It provides an interactive environment for solving problems involving linear algebra, calculus, statistics, optimization, signal processing, and numerical methods.

GNU Octave is largely compatible with MATLAB, making it an excellent alternative for learning and research purposes without licensing costs. Programs written in MATLAB can often be run in Octave with little or no modification.

Octave supports matrix-based computation, which allows users to work efficiently with vectors, matrices, and arrays. It also includes powerful plotting and visualization tools with built-in 2D/3D plotting, to represent data graphically.

Installation of Octave

To install GNU Octave, a free and open-source numerical computation software, go to the official GNU Octave website www.octave.org or refer the following link

For 32 bit: <https://www.fileeagle.com/software/download/16865/d143f3>

For 64 bit: <https://octave.org/download>

Installation Procedure of GNU Octave (Steps)

1. **Open the official website**

Go to the Octave download page (www.octave.org)

2. **Download the installer**

Select the installer suitable for your operating system (Windows/Linux/Mac)

Or Check your system type (**32-bit or 64-bit**)

Download the appropriate installer (32-bit or 64-bit).

For 32 bit: <https://www.fileeagle.com/software/download/16865/d143f3>

For 64 bit: <https://octave.org/download>

3. **Run the installer**

Double-click the downloaded setup file.

4. **Follow installation steps**

Click **Next** → **Accept license** → **Choose location** → **Install**

5. **Complete installation**

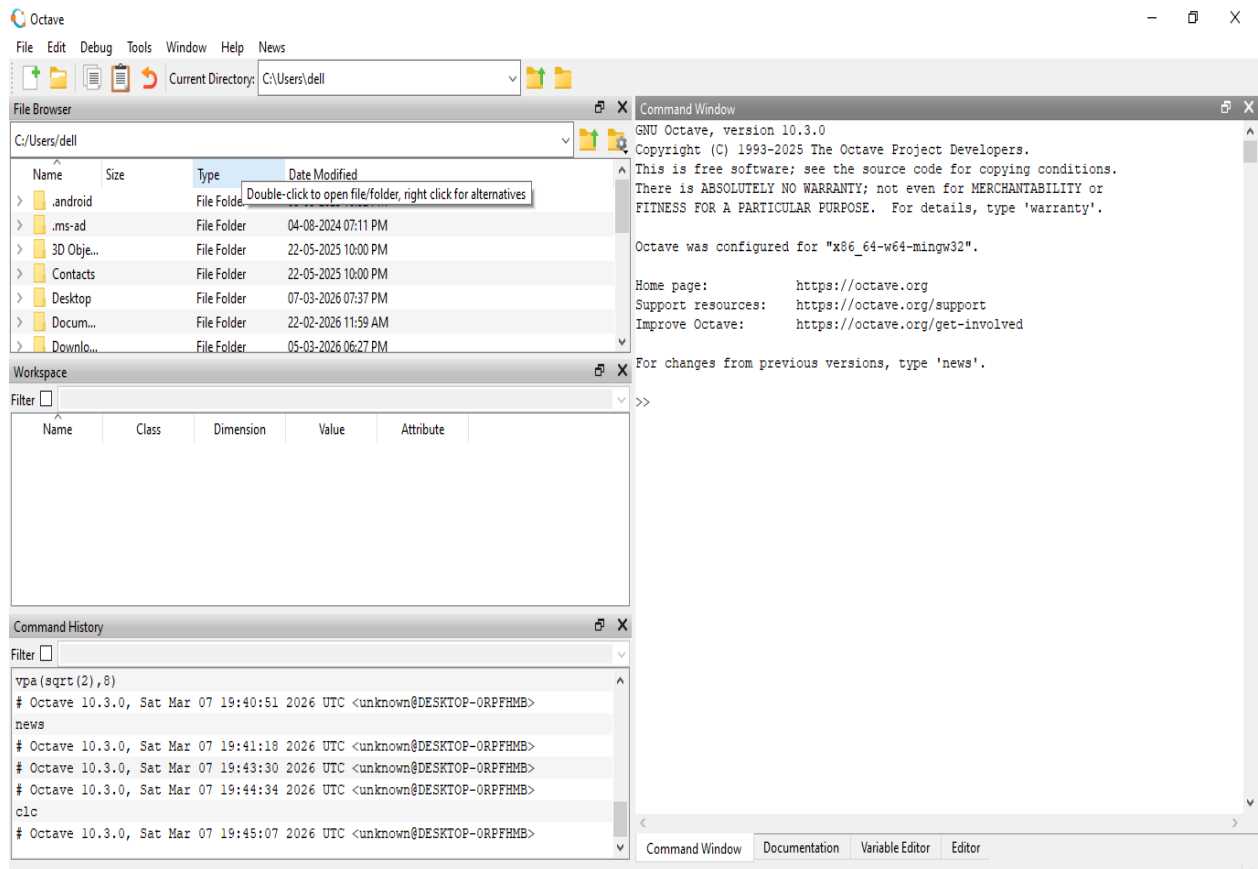
Click **Finish** after installation is completed.

6. **Start Octave**

Open **GNU Octave** from the **Start menu / Applications**

Result: Octave opens with its interface (Command Window, Editor, Workspace, etc.) ready for use

GNU Octave Interface Overview



Component	Meaning	Purpose / Function
File Browser	Window used to view files and folders in the current directory	Helps to open, manage, and organize script files (.m files)
Workspace	Area that displays all variables currently stored in memory	Shows variable names, values, size, and type during program execution
Command History	List of previously executed commands	Allows users to reuse or run previous commands easily
Command Window	Main interface where users type commands and see results	Executes commands, performs calculations, and displays outputs or errors
Documentation	Built-in help system of Octave	Provides information about commands, functions, syntax, and examples
Variable Editor	Tool used to view and edit variable values in table form	Helps users inspect and modify variables stored in the workspace
Editor	Window used to write and edit program files	Allows creating, modifying, and saving Octave scripts and functions

Step-by-Step: How to Run a Symbolic Program in GNU Octave

Step 1 — Open GNU Octave

- Launch GNU Octave GUI on your computer.
- You'll see the Command Window, Editor, and Workspace panels.

Step 2 — Load the Symbolic Package

In the Command Window, type:

```
pkg load symbolic
```

If you see an error like *"package not found"*

install it first:

```
pkg install -forge symbolic
```

```
pkg load symbolic
```

Note: You only need to install once — after that, pkg load symbolic is enough each time.

Open the Editor and Create a New Program:

1. Click File → New → Script (or press Ctrl + N).
2. Type or paste your program — for example:

```
pkg load symbolic;  
syms x;  
f = input('Enter f(x): ');  
df = diff(f, x);  
disp('Derivative of f(x) = ');  
disp(df);
```

Save the file with a .m extension, e.g.

derivative_symbolic.m

****Make sure to save it in the current working directory (check path in top bar or with pwd).**

Run the Program There are two ways:

- (a) From Command Window Type the file name without.m:
derivative_symbolic
- (b) From the Editor Click the Run (green triangle) button on the toolbar.

Give Runtime Input

Octave will ask for input at runtime.

Example:

Enter f(x): $\sin(x)^2 + 3x$

Press **Enter** — Octave will process symbolically and display results

View Output

Example Output:

Derivative of $f(x) = 2\sin(x)\cos(x) + 3$

LAB 1: Program to compute addition and subtraction of matrices

%Program to compute addition and subtraction of matrices

```
clc;
clear;
A = input('Enter matrix A ');
B = input('Enter matrix B ');
if size(A)==size(B)
    C = A + B;
    D = A - B;
    disp('Addition of two matrices:');
    disp(C);
    disp('Subtraction of two matrices:');
    disp(D);
else
    disp('Order of matrices are not equal');
end
```

Output:

Enter matrix A [-2,6,8;7,4,-5;-2,5,9]

Enter matrix B [-3,-2,6;3,-4,-9;5,8,7]

Addition of two matrices:

-5 4 14
10 0 -14
3 13 16

Subtraction of two matrices:

1 8 2
4 8 4
-7 -3 2

Example: Compute addition and subtraction of matrices

$$A = \begin{bmatrix} -2 & 6 & 8 \\ 7 & 4 & -5 \\ -2 & 5 & 9 \end{bmatrix} \text{ and}$$

$$B = \begin{bmatrix} -3 & -2 & 6 \\ 3 & -4 & -9 \\ 5 & 8 & 7 \end{bmatrix}$$

LAB 2: Program to compute rank of matrix and row reduced echelon form.

% Program to find rank and RREF of a matrix

clc;

A = input('Enter the matrix A ');

% Row Reduced Echelon Form

B = rref(A);

% Rank of matrix

r = rank(A);

disp('Row Reduced Echelon Form of A:');

disp(B);

disp('Rank of matrix A:');

disp(r);

Output:

Enter the matrix A [2 4 6;1 3 5;0 1 2]

Row Reduced Echelon Form of A:

1 0 -1

0 1 2

0 0 0

Rank of matrix A:

2

Example: Find the RREF and

$$\text{Rank of } A = \begin{bmatrix} 2 & 4 & 6 \\ 1 & 3 & 5 \\ 0 & 1 & 2 \end{bmatrix}$$

LAB 3: Program to find the angle between the radius vector and tangent

%Program to find the angle between the radius vector and tangent

clc;

pkg load symbolic;

% Declare symbolic variable

syms t a

% Define polar function

r=input("Enter the function : ");

%example a*(1 + cos(t));

% Differentiate w.r.t t

dr = diff(r, t);

% Read theta as STRING and convert directly to symbolic

t0 = sym(input('Enter value of theta : ', 's'));

% Compute tan(psi)

s=simplify(atan(r / dr));

% Compute angle between radius vector and tangent at t=t0

psi=subs(s, t, t0);

if psi < 0

psi = pi - abs(psi);

end

disp('Angle between radius vector and tangent is ');

disp(simplify(psi));

Output:

Enter the function : a*(1+cos(t))

Enter value of theta : pi/2

Angle between radius vector and tangent is

$$\frac{3\pi}{4}$$

Example: Find the angle between the radius vector and the tangent for the curve $r = a(1 + \cos(\theta))$ at $\theta = \frac{\pi}{2}$

LAB 4: Program to find the n^{th} derivative of e^{ax} and trigonometric functions

%Program to find the nth derivative of the given function

clc;

clear;

pkg load symbolic;

% Define symbolic x for display

syms x

% Input values

y=input('Enter the function : ');

% example exp(2*x+3),sin(x+3);

n = input('Enter order of derivative : ');

% General formula for nth derivative

yn=diff(y,x,n);

% Display result

printf('The derivative of the given function y%d is\n',n);

disp(yn);

Output:

Enter the function : exp(2*x+3)

Enter order of derivative : 3

The derivative of the given function y3 is

$$8 * e^{2 * x + 3}$$

Example: Find the 3rd Derivative of e^{2x+3}

LAB 5: Program to construct Cayley table and test Commutative for given finite set

%Program to construct Cayley table and test commutative law for given finite set.

```
clc;
clear;
pkg load symbolic
S=input("Enter group elements as row vector: ");
k=length(s);
bo=@(a,b)mod(a+b,4);
T=zeros(k,k);
for i=1:k
    for j=1:k
        T(i,j)=bo(S(i),S(j));
    endfor
endfor
disp("Cayley table is : ");
disp(T);
if isequal(T,T.')
    disp("Given binary operation is commutative");
else
    disp("Given binary operation is not commutative");
endif
```

Output:

Enter group elements as row vector: [0,1,2,3]

Cayley table is :

0	1	2	3
1	2	3	0
2	3	0	1
3	0	1	2

Given binary operation is commutative

% program using cartprod()

```
clc;
clear;
pkg load linear-algebra;
% Define the set
S = [0 1 2 3]
k = length(S);
% Cartesian product of  $S \times S$ 
P = cartprod(S,S);
% Binary operation  $(a+b) \bmod 3$ 
R = mod(P(:,1) + P(:,2),4);
% Convert result into matrix form
T = reshape(R,k,k);
disp("Cayley Table is")
disp(T)
% Check Abelian property
if isequal(T, T.')
    disp("Given binary operation is commutative");
else
    disp("Given binary operation is not commutative");
endif
```

Example: Write the Cayley table for $G=\{0,1,2,3\}$ under addition modulo 4 and check the commutative law.

ii) Program to construct Cayley table and test Commutative for given finite set

%Program to construct Cayley table and test commutative law for given finite set.

```
clc;
clear;
pkg load symbolic
S=input("Enter group elements as row vector: ");
k=length(S);
bo=@(a,b)(a*b);
T=zeros(k,k);
for i=1:k
    for j=1:k
        T(i,j)=bo(S(i),S(j));
    endfor
endfor
disp("Cayley table is : ");
disp(T);
if isequal(T,T.')
    disp("Given binary operation is commutative");
else
    disp("Given binary operation is not commutative");
endif
```

Output:

Enter group elements as row vector: [1,-1,i,-i]

Cayley table is :

1 + 0i	-1 + 0i	0 + 1i	0 - 1i
-1 + 0i	1 + 0i	0 - 1i	0 + 1i
0 + 1i	0 - 1i	-1 + 0i	1 + 0i
0 - 1i	0 + 1i	1 + 0i	-1 + 0i

Given binary operation is commutative

% program using cartprod()

```
clc;
clear;
pkg load linear-algebra;
% Define the set
S = [1 1 i -i]
k = length(S);
% Cartesian product of S x S
P = cartprod(S,S);
% Binary operation (a*b)
R = P(:,1) .* P(:,2);
% Convert result into matrix form
T = reshape(R,k,k)';
disp("Cayley Table is")
disp(T)
% Check Abelian property
if isequal(T, T.')
    disp("Given binary operation is commutative");
else
    disp("Given binary operation is not commutative");
end
```

Example: Construct Cayley table and test commutative for $G=\{1, -1, i, -i\}$

```
S =
    1 + 0i    -1 + 0i     0 + 1i     0 - 1i

Cayley Table is
    1 + 0i    -1 + 0i     0 + 1i     0 - 1i
   -1 + 0i     1 - 0i     0 - 1i     0 + 1i
    0 + 1i     0 - 1i    -1 + 0i     1 - 0i
    0 - 1i     0 + 1i     1 - 0i    -1 + 0i
Given binary operation is commutative
```

LAB 6: Programs to verify Lagrange's theorem (groups) with suitable example

% Program to verify Lagrange's theorem

```
clc;
clear;
pkg load linear-algebra;
G = input('Enter group elements as row vector: ');
H = input('Enter subgroup elements as row vector: ');
m=length(H);
n=length(G);
bo=@(a,b)(a*b);
if ~all(ismember(H,G))
    disp('H is not a subset of G');
else
    H1=cartprod(H,H);
    i=1;
    for a=1:m*m
        H2(a)=bo(H1(a,1),H1(a,2));
    endfor
    H3=unique(H2)
    printf('order(G)=%d, order(H)=%d\n',n,m);
    if all(ismember(H3,H))
        if mod(n,m)==0
            disp('Lagrange's theorem verified');
        else
            disp('Lagrange's theorem not satisfied');
        end
    else
        disp('not a subgroup');
    end
end
```

Output:

```
Enter group elements as row vector: [1,-1,i,-i]
Enter subgroup elements as row vector: [1,-1]
order(G)=4, order(H)=2
Lagrange's theorem verified
```

% program other way

```
clc;
clear;
pkg load linear-algebra;
G = [1 -1 i -i]
H = [1 -1]
n = length(G);
m = length(H);
disp(['Order of G = ', num2str(n)])
disp(['Order of H = ', num2str(m)])
% Check if H is subset of G
if all(ismember(H,G))
    % Cartesian product
    P = cartprod(H,H);
    % Operation result
    R = unique(P(:,1).*P(:,2));

    % Check closure
    if all(ismember(R,H))
        if mod(n,m)==0
            disp('H is a subgroup of G and
Lagrange's theorem is satisfied')
        else
            disp('H is a subgroup but Lagrange's
theorem not satisfied')
        end
    else
        disp('H is not a subgroup of G')
    end
else
    disp('H is not a subset of G')
end
```

Example: verify Lagrange's theorem

$G = \{1, -1, i, -i\}$ be the group under multiplication and $H = [1, -1]$ be the subgroup of G

LAB 7: Program to evaluate the double and triple integrals with constant limits

%Program to evaluate the double and triple integrals with constant limits

clc;

clear;

pkg load symbolic;

% Declare symbolic variables

syms x y z

% Input function

f = input('Enter the function f : ');

choice = input('Enter 2 for double or 3 for triple integral: ');

% Input limits

a = input('Enter lower limit of x : ');

b = input('Enter upper limit of x : ');

c = input('Enter lower limit of y : ');

d = input('Enter upper limit of y : ');

if choice == 2

I = int(int(f, x, a, b), y, c, d);

disp('The value of the double integral is:');

elseif choice==3

e = input('Enter lower limit of z : ');

g = input('Enter upper limit of z : ');

% Evaluate tripple integral

I= int(int(int(f, x, a, b), y, c, d),z,e,g);

disp('The value of the tripple integral is:');

end

disp(I);

Example: Evaluate $\int_0^1 \int_0^2 x + y \, dy dx$

Output:

Enter the function f : x+y

Enter 2 for double or 3 for triple integral: 2

Enter lower limit of x : 0

Enter upper limit of x : 1

Enter lower limit of y : 0

Enter upper limit of y : 2

The value of the double integral is:

3

LAB 8: Program to verify the exactness of a differential equation

%Program to verify the exactness of a differential equation

clc;

clear;

pkg load symbolic;

% Declare symbolic variables

syms x y

% Input M and N

M = input('Enter M(x,y): ');

N = input('Enter N(x,y): ');

% Partial derivatives

My = diff(M, y);

Nx = diff(N, x);

% Display derivatives

disp('diff(m,y) =');

disp(My);

disp('diff(n,x) =');

disp(Nx);

% Check exactness

if My==Nx

disp('The given differential equation is exact');

else

disp('The given differential equation is not exact');

end

Example: Test whether the differential equation is exact
 $(y^2 e^{xy^2} + 4x^3)dx + (2xye^{xy^2} - 3y^2)dy = 0$ is exact.

Output:

```
Enter M(x,y) : y^2*exp(x*y^2)+4*x^3
Enter N(x,y) : 2*x*y*exp(x*y^2)-3*y^2
diff(m,y) =
      2      2
      3 x*y      x*y
      2*x*y *e  + 2*y*e
diff(n,x) =
      2      2
      3 x*y      x*y
      2*x*y *e  + 2*y*e
The given differential equation is exact
```

LAB 9: Program to find the Complementary Function and Particular Integral of linear differential equations with constant coefficients.

% Program to find the CF and PI of linear differential with constant coefficients

clc;

clear;

pkg load symbolic;

syms y(x)

y1=diff(y,x);

y2=diff(y,x,2);

% Input differential equation

eqn = input('Enter the differential equation : ');

% Solve the differential equation

sol = dsolve(eqn);

CF=dsolve(lhs(eqn));

PI=simplify(sol-CF);

% Display result

printf('CF = %s\n',char(CF));

printf('PI = %s\n',char(PI));

printf('The general solution y= %s\n',char(CF+PI));

Example: Solve the differential equation

$$\frac{d^2y}{dx^2} - 9\frac{dy}{dx} + 18y = e^{-3x}$$

or $y_2 - 9y_1 + 18y = e^{-3x}$

Output:

Enter the differential equation : $y_2 - 9y_1 + 18y = \exp(-3x)$

CF = $(C1 + C2 \cdot \exp(3x)) \cdot \exp(3x)$

PI = $\exp(-3x)/54$

The general solution y= $(C1 + C2 \cdot \exp(3x)) \cdot \exp(3x) + \exp(-3x)/54$

LAB 10: Program to verify D'Alembert's and Raabe's tests with suitable examples

```
%Program to verify D'Alembert's and Raabe's tests
clc;
clear;
pkg load symbolic;
syms n
u=input('Enter the general term un: ');
u=matlabFunction(u);
% D'Alembert's Test
L=limit(u(n+1)/u(n),n,inf);
disp('Using D"Alembert Test L=');
disp(L);
if L < 1
disp('By D"Alembert ratio test, the series is convergent');
elseif L > 1
disp('By D"Alembert ratio test, the series is divergent');
else
disp('D"Alembert ratio test fails');
% Raabe's Test
L = limit(n*(u(n)/u(n+1) - 1), n, inf);
disp('Using Raabes Test L=');
disp(L);
if L > 1
disp('By Raabes test The series is convergent ');
elseif L < 1
disp('By Raabes test the series is divergent');
else
disp('Raabes test fails');

end
end
```

Example: Check the convergence of $\sum_{n=1}^{\infty} \frac{1}{n!}$

Output:

Enter the general term un: 1/factorial(n)
Using D'Alembert Test L=0
By D'Alembert ratio test, the series is

LAB 11: Program to find the Laplace transforms of some standard functions.

%Program to find the Laplace transforms of some standard functions

clc;

clear;

pkg load symbolic

syms t S

% Input function

f = input('Enter the function f(t): ');

% Example: exp(2*t), sin(3*t), t^2

% Laplace Transform

F = laplace(f, t, S);

disp('Laplace Transform is:');

disp(F);

Output:

Enter the function f(t): sin(3*t)

Laplace Transform is:

$$\frac{3}{s^2+9}$$

Example: Evaluate $L\{\sin(3t)\}$



LAB 11: Program to find the Laplace transforms of periodic functions.

% Program for Laplace transform of periodic functions

clc;

clear;

pkg load symbolic;

% Declare symbols

syms t S

assume S positive

% Input periodic function and period

f = sym(input('Enter the periodic function f(t): ', 's'));

T = sym(input('Enter the period T: ', 's'));

% Laplace transform using periodic formula

F = int(exp(-S*t)*f, t, 0, T) / (1 - exp(-S*T));

disp('Laplace Transform of the periodic function is:');

disp(F)

Example: If $f(t) = t$ and $f(t + 1) = f(t)$,
 $0 \leq t < 1$ then find $L\{f(t)\}$.

Output:

```
Enter the periodic function f(t): t
Enter the period T: 1
Laplace Transform of the periodic function is:
```

$$\frac{(-S - 1)e^{-S} + \frac{1}{S^2}}{1 - e^{-S}}$$

LAB 12: Program to find sine and cosine Fourier transforms

%Program to find sine and cosine Fourier transforms.

clc;

clear;

pkg load symbolic;

syms x f l n

assume n integer positive

% Input function

f = sym(input('Enter the function f(x): ','s'));

% Example: exp(-x)

l=input("Enter the value of l : ");

% Fourier Sine Transform

Fs = int(f*sin(n*pi*x/l), x,0,l);

% Simplify the result

Fs = simplify(Fs);

disp('Fourier Sine Transform FS(n) is:');

disp(Fs);

% Fourier Cosine Transform

Fc = int(f*cos(n*pi*x/l), x,0,l);

Fc0 = int(f, x,0,l);

disp('Fourier Cosine Transform FC(0) is:');

disp(Fc0);

disp('Fourier Cosine Transform FC(n) is:');

disp(Fc);

Example: Find the finite sine and cosine transform of $f(x) = x$ in the interval $(0, \pi)$.

Output

```
Enter the function f(x): x
Enter the value of l : pi
Fourier Sine Transform FS(n) is:
      n + 1
  (-1)      *pi
  -----
      n
Fourier Cosine Transform FC(0) is:
      2
  pi
  ---
      2
Fourier Cosine Transform FC(n) is:
      n
  (-1)      1
  ----- - ---
      2      2
      n      n
```

Summary of commands

Command	Meaning
;	semicolon is used to suppress the output of a command in the command window
assume variable condition	It is used to assign a condition or property to a symbolic variable. Or sets condition on variable It means telling Octave that a symbolic variable satisfies a specific condition, such as being positive, negative, nonnegative, nonpositive, even, odd, real, or an integer.
cartprod(A,B)	Cartesian product of sets(All ordered pairs)
cd('path')	Change directory
char()	Convert symbolic expression $\rightarrow$ string char() converts numeric ASCII codes or symbolic expressions into characters (strings) or converts values or symbolic expressions into character strings.
clc	Clear command window
clear / clear all	Clear workspace
diff(f) diff(f, x) diff(f, x, n)	Derivative of f with respect to x. It is used to find the derivative of a function. nth order derivative of f with respect to x
disp(x)	Display the value of x (It is used to display text messages or variable values on the screen)
double()	converts a symbolic expression to double precision numeric value, whereas vpa() evaluates it with variable (higher) precision or converts symbolic value to normal numeric value
dsolve(equation) dsolve(equation, condition) equation $\rightarrow$ differential equation condition $\rightarrow$ initial/boundary condition (optional)	dsolve() finds the symbolic solution of an ordinary differential equation (ODE).
expand(expr)	Expands algebraic expression
factor(expr)	Factorizes polynomial
int(f) int(f,x) int(f,x,a,b)	indefinite integral Integral f of with respect to x Or integrate f with respect to x definite integral f from a to b.
intersect(A,B)	Finds common elements of two sets
ismember(A, B) A $\rightarrow$ elements to check B $\rightarrow$ reference set Checks whether elements belong to another set	Checks if elements of A belong to B. Checks whether elements of one array/set are present in another array/set. It returns 1 (true) if the element is present and 0 (false) if it is not

Command	Meaning
all(ismember(A,B)) $A \subseteq B$ if every element of A is in B	subset checking can be done using functions like ismember()
laplace(f, t, S) f $\rightarrow$ function of time t t $\rightarrow$ original variable (time) S $\rightarrow$ Laplace variable	computes the Laplace transform of function from variable t to s. $L\{f(t)\}=F(S)$
length(A)	Number of elements in a set/vector
lhs(eqn) eqn $\rightarrow$ symbolic equation	lhs() returns the left-hand side of a symbolic equation
limit(f,x,a)	Finds limit f as $x \rightarrow a$
ls	List files
matlabFunction() matlabFunction(expr) matlabFunction(expr,'vars',[var1 var2 ...]) expr $\rightarrow$ symbolic expression vars $\rightarrow$ variables used in the function	Specifies input variables for the function Specifies input variables for the function It is used to convert a symbolic expression into a MATLAB function handle in MATLAB Converts symbolic expression to function Specifies input variables for the function
mod(a, b)	The function mod() is used to find the remainder after division. Returns the remainder when a is divided by b. Same sign as divisor (b) $\text{mod}(a,b)=a-b \times \text{floor}(a/b)$ $\text{floor}(x) \rightarrow$ Returns the largest integer less than or equal to x. $\text{floor}(x) \leq x$
num2str(number)	converts a numeric value into a string so that it can be displayed with text or Convert numeric value $\rightarrow$ string
pkg load symbolic pkg load linear-algebra	Load the symbolic package (Symbolic mathematics (algebra & calculus) functions) Load the linear algebra package (Package that contains advanced linear algebra functions)
fprintf('format text', variables)	fprintf() is used to display formatted output in the command window. Or printf() prints formatted text and variable values. Format specifier: %s for string, %d for integer, %f for floating point.
pwd	Show current directory

Command	Meaning
rem(a,b)	It is used to compute the remainder after division. It returns the remainder of division and the result has the same sign as the dividend. Same sign as dividend (a) $\text{rem}(a,b)=a-b \times \text{fix}(a/b)$ $\text{fix}()$ → rounds toward zero or moves toward zero $\text{fix}(x)$ → Returns the integer part of x by removing the decimal part.
reshape(A, m, n)	Rearranges the elements of A into a matrix with m rows and n columns. The function reshape() is used to change the size (dimensions) of a matrix or vector without changing its elements. It rearranges the elements of a vector or matrix into a matrix of specified size without changing the total number of elements. The total number of elements must remain the same
setdiff(A,B)	Elements in A but not in B
simplify(expr)	Simplifies algebraic expression
solve(eq,x)	Solves equation for variable x
sort(A) sort(A, mode) mode → "ascend" or "descend"	Arranges elements in ascending order
subs(expression,variable, value)	It is used in symbolic mathematics to substitute a value or expression for a variable in a symbolic expression or It replaces a symbolic variable with a specific value or another expression
sym(value)	Converts expression to symbolic
syms x y t	Declare symbolic variables
union(A,B)	Combines elements of two sets without repetition
unique(A)	Removes duplicate elements
var=input(prompt)	Display prompt and wait for user input
var=input(prompt,'s')	The character string 's' as the second argument, This tells Octave to return the string entered by the user directly (or read the input as a string)
vpa()	Variable precision arithmetic or gives high precision decimal value